Streamlit API 핵심 요약 및 활용 가이드

1. Streamlit : 데이터 스크립트를 웹 앱으로 전환하기

Streamlit은 데이터 스크립트를 단 몇 분 만에 공유 가능한 웹 앱으로 변환해 줍니다. 복잡한 프런트엔드 지식 없이도 데이터를 시각화(Visualize), 변형(Mutate), 공유(Share)할 수 있는 가장 빠른 방법입니다.

2. 화면 출력의 기초 : Write와 Magic

Streamlit에서 화면을 구성하는 가장 기본은 st.write와 Magic입니다.

st.write : '스위스 아미 나이프(Swiss Army Knife)'로 불리며, 데이터프레임, 차트, 마크다운 등 거의 모든 객체를 알아서 렌더링합니다. 다만 프로덕션 환경에서 더 정밀한 레이아웃 제어와 성능 최적화를 위해 st.dataframe이나 st.plotly_chart같은 전용 함수를 사용하는 것을 권장합니다.

Magic : 변수나 리터럴을 코드 한 줄에 적기만 하면 자동으로 출력해주는 편리한 기능입니다.

구분	st.write	Magic
명시성	명시적으로 함수를 호출함	코드 줄에 값만 적음 (암시적)
유연성	여러 인자를 한 번에 출력 가능	단일 변수/값 출력에 최적화
제한 사항	어디서나 사용 가능	복잡한 제어 흐름(함수 내부 등)에서는 작동하지 않을 수 있음
추천 사례	명확한 의도 전달이 필요한 코드	빠른 디버깅 및 초기 프로토타이핑

```
import streamlit as st
import pandas as pd

df = pd.DataFrame({'id': [1, 2], 'value': [10, 20]})

st.write("### 데이터 확인", df) # st.write 사용
df # Magic 사용 (화면에 데이터프레임이 바로 나타남)
```

3. 텍스트 요소: 정보의 계층 구조 설계

텍스트 API는 앱의 가독성과 정보 전달력을 결정짓는 핵심 요소입니다.

- **st.title:** 앱의 메인 제목. 페이지당 하나만 사용하는 것이 SEO와 UX 측면에서 좋습니다.
- **st.header / st.subheader:** 섹션과 하위 섹션을 나누는 계층 구조를 만듭니다.
- **st.markdown:** 마크다운 문법을 지원하며, 텍스트 스타일링에 가장 많이 쓰입니다.
- **st.caption:** 데이터 출처나 주석을 위해 작은 폰트로 출력합니다.
- **st.latex:** 복잡한 수식을 미려하게 표현합니다.
- **st.code vs st.echo:**
    - **st.code:** 특정 코드 블록을 정적으로 보여줍니다. (코드 강조용)
    - **st.echo:** 코드 블록을 감싸서 **코드 자체와 실행 결과**를 동시에 보여줍니다. 시니어 개발자들은 사용법을 설명하는 **튜토리얼 제작 시** 이 기능을 적극 활용합니다.

```
st.title("앱의 메인 제목")
st.header("섹션")
st.subheader("하위섹션")
st.markdown("텍스트 스타일링")
st.caption("출처나 주석을 위한 작은 폰트")
```

4. 데이터 및 차트 요소: 데이터의 시각적 통찰력 강화

데이터 시각화는 Streamlit의 백미입니다. 단순 조회를 넘어 편집까지 가능합니다.

- **st.metric**: 매출, 사용자 수 등 핵심 지표(KPI)를 표시하며, **delta** 파라미터로 증감을 강조할 때 유용합니다.
- **st.map**: 지도 시각화 도구입니다.
- **중요:** Streamlit은 데이터프레임에서 자동으로 **'lat'/'latitude'** 및 **'lon'/'longitude'**라는 이름의 컬럼을 찾아 매핑합니다. 이 컬럼명이 다르면 지도가 표시되지 않으니 주의하십시오.

기능	st.dataframe	st.table	st.data_editor
상태	인터랙티브 (정렬, 검색)	정적 (고정된 표)	**인터랙티브 + 편집 가능**
용도	대량 데이터 탐색	요약 정보 전시	**데이터 수정 및 입력**

```python
# st.map 예시
import pandas as pd
import numpy as np

# 'lat'과 'lon'이라는 컬럼명이 필수입니다.
map_data = pd.DataFrame(
    np.random.randn(100, 2) / [50, 50] + [37.56, 126.97],
    columns=['lat', 'lon'])

st.map(map_data)
```

5. 입력 위젯: 사용자 상호작용의 구현

위젯은 사용자의 입력을 파이썬 변수로 즉시 연결해 줍니다.

입력 타입	API명	반환 값(Return)	주요 용도
클릭	`st.button`	`True` / `False`	로직 실행, 데이터 새로고침
논리	`st.checkbox`	`True` / `False`	특정 기능/데이터 활성화
선택(단일)	`st.selectbox`	선택된 객체(str, int 등)	드롭다운 메뉴에서 옵션 선택
선택(다중)	`st.multiselect`	선택된 객체들의 리스트	여러 태그나 조건 다중 필터링
수치	`st.slider`	선택된 수치(int, float)	범위 지정 필터링

```
# 슬라이더를 이용한 데이터 필터링 로직
# 사용자가 슬라이더를 움직이면 'selected_year' 변수가 즉시 업데이트되고 앱이 재실행됩니다.
selected_year = st.slider('연도 선택', 2015, 2023, 2020)
st.write(f"선택된 연도: {selected_year}년 데이터입니다.")
```

6. 레이아웃 및 컨테이너: 앱의 사용자 경험(UX) 최적화

복잡한 위젯들을 질서 있게 배치하는 것은 앱의 완성도를 좌우합니다.

1. **st.sidebar**: 화면 왼쪽의 고정 영역입니다. 시니어 개발자들은 주로 **"글로벌 컨트롤(필터링, 설정)"**을 배치하여 메인 화면이 결과물에 집중할 수 있도록 구성합니다.
2. **st.columns**: 화면을 가로로 분할합니다. with 문(Context Manager)을 사용하여 각 컬럼 내 요소를 독립적으로 정의하는 것이 표준 패턴입니다.
3. **st.tabs / st.expander**: 공간을 절약하고 정보를 계층적으로 숨기거나 보여줄 때 유용합니다.

```python
# 컬럼 배치 베스트 프랙티스
col1, col2 = st.columns([2, 1]) # 2:1 비율로 분할
with col1:
    st.header("메인 차트 영역")
    st.line_chart([1, 5, 2, 6])
with col2:
    st.header("상세 정보")
    st.write("차트에 대한 세부 설명입니다.")
```

7. 실행 흐름 및 성능 최적화: Caching과 State

Streamlit 앱은 사용자가 위젯을 조작할 때마다 **전체 스크립트를 처음부터 끝까지 재실행**합니다.

- **Caching 매커니즘**: @st.cache_data를 사용하면 Streamlit은 **함수의 코드**와 **입력 인자**를 해싱(Hashing)합니다. 만약 이 두 가지가 변하지 않았다면, 함수를 실행하지 않고 로컬 캐시에서 결과값을 가져옵니다.

Pro-Tip: 데이터 처리는 @st.cache_data를 사용하고, 데이터베이스 연결이나 머신러닝 모델 로드와 같은 무거운 리소스는 @st.cache_resource를 사용하십시오.

Caching의 3가지 제한 사항 (Source Context 기반):

1. **작업 디렉토리**: 현재 작업 디렉토리 외부의 라이브러리 변경 사항은 감지하지 못할 수 있습니다.
2. **비결정적(Non-deterministic) 소스**: 랜덤 값이나 실시간 시세처럼 호출 시점마다 변하는 값은 캐시하기 부적절합니다.
3. **변형(Mutation)**: 캐시된 결과는 참조(Reference)로 저장되므로, 함수 밖에서 반환된 객체를 직접 수정하는 것은 피해야 합니다.

```python
# Text 출력
@st.cache_data
def change_text():
    text = st.title("제일 큰 글자")
    time.sleep(0.5)
    text.header("title 보다 작은 글자")
    time.sleep(0.5)
    text.subheader("일반 글자보다 약간 큼")
    time.sleep(0.5)
    text.text("일반 텍스트")
change_text()
```

8. 상태 요소 및 유틸리티: 완성도 높은 보고서형 앱 마무리

앱이 동작 중임을 사용자에게 알리는 시각적 피드백은 UX의 핵심입니다.

- **st.progress / st.spinner**: 데이터 로딩이나 복잡한 연산 중에 사용자의 이탈을 방지합니다.
- **상태 알림 가이드**:
    - st.success (초록색): 데이터 처리 성공, 배포 완료 알림
    - st.info (파란색): 일반 가이드 및 정보 전달
    - st.warning (노란색): 데이터 결측치 주의, 설정 변경 권고
    - st.error (빨간색): API 연결 실패, 잘못된 입력 값 안내

```python
# 특정 상황에 사용하는 Text
st.success("성공")
st.warning("경고")
st.info("정보")
st.error("오류")
```

```python
if submitted:
    st.text(f"이름 : {name} , 나이 : {age}")
    if agree:
        st.success("약관에 동의했습니다.")
    else:
        st.warning("약관에 동의하지 않았습니다.")
```